

习题 11

一、选择题

1. 在 C++ 中,模板的主要用途是()。
A. 提高程序运行时的效率
B. 编写与数据类型无关的代码
C. 减少代码的可读性
D. 限制代码的复用性
2. 下列()关键字用于声明模板。
A. typedef
B. inline
C. template
D. constexpr
3. 在模板参数列表中,通常使用()关键字来指定类型参数。
A. Class
B. struct
C. typename
D. enum
4. 函数模板可以处理的参数类型是()。
A. 只有基本数据类型
B. 只有用户自定义类型
C. 基本数据类型和用户自定义类型
D. 不能处理任何类型的参数
5. 以下()是 C++ 标准库中的动态数组容器。
A. array
B. list
C. vector
D. map
6. `std::vector` 和 `std::array` 的主要区别是()。
A. `std::vector` 可以动态改变大小,而 `std::array` 大小固定
B. `std::array` 可以动态改变大小,而 `std::vector` 大小固定
C. 两者都可以动态改变大小
D. 两者大小都固定
7. 在 C++ 中,连接两个字符串的方法是()。
A. 使用 `+` 运算符
B. 使用 `&` 运算符
C. 使用 `concat()` 函数
D. 使用 `append()` 方法
8. 模板类和非模板类的最主要的区别是()。
A. 模板类可以处理不同类型的数据
B. 非模板类可以处理不同类型的数据
C. 模板类使用更多的内存
D. 非模板类使用更多的内存
9. 在 C++ 中,声明一个函数模板的方法是()。
A. 使用关键字 `function`
B. 使用关键字 `template`
C. 使用关键字 `def`
D. 使用关键字 `fun`
10. 下列()是 `std::string` 类的构造函数。
A. `string(const char * s)`
B. `string(std::initializer_list<char>)`
C. `string(size_t n, char c)`
D. 所有选项都是

二、写出程序的运行结果

1.

```
#include <iostream>
#include <vector>
#include <string>

using namespace std;

//学生信息类模板
template<typename T>
class StudentInfo {
public:
    string name;
    T score;

    StudentInfo(string n,T s):name(n),score(s){}

    void display() const {
        cout <<"Name:"<< name <<" , Score:"<< score << endl;
    }
};

int main(){
    vector<StudentInfo<int>> students;
    students.emplace_back("Alice",90);
    students.emplace_back("Bob",85);
    students.emplace_back("Charlie",95);

    for(const auto& student:students){
        student.display();
    }

    return 0;
}
```

2.

```
#include <iostream>

using namespace std;

//函数模板计算平均值
template<typename T>
T calculateAverage(T a,T b)
{
    return (a + b)/2;
}

int main()
{
    int result1 = calculateAverage(10,20);
```

```

        double result2 = calculateAverage(15.5, 24.5);

        cout << "Average of integers:" << result1 << endl;
        cout << "Average of doubles:" << result2 << endl;

        return 0;
    }

```

3.

```

#include <iostream>
#include <vector>
#include <string>

using namespace std;

template <typename T>
void printVector(const vector<T> & v)
{
    for(const auto& item:v)
    {
        cout << item << " ";
    }
    cout << endl;
}

int main()
{
    vector<int> numbers = { 1, 2, 3, 4, 5 };
    printVector(numbers);

    vector<string> names = { "Alice", "Bob", "Charlie" };
    printVector(names);

    return 0;
}

```

三、程序填空

1. 设计模板函数 Sort() 和 Print(), 其中 Sort() 能对给定的数据进行排序, Print() 能打印输出数组, 模板参数 T 可实例化成字符串, 运行下列程序后可得到如图 11.3 所示的运行结果。

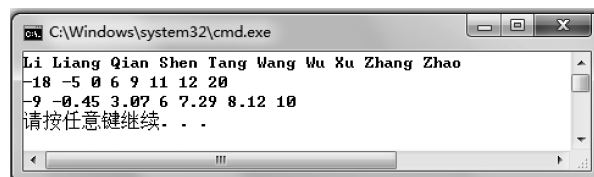


图 11.3 程序填空题 1 运行结果

```

#include <iostream>
#include <string>

```

```

using namespace std;
    (1)
{   int i, j;
    (2)
    for(i = 0; i < n - 1; i++)
        for(j = 0; j < n - i - 1; j++)
            if( (3) )
                {   t = a[j];
                    a[j] = a[j + 1];
                    a[j + 1] = t;
                }
}
    (4)
{   int i;
    for(i = 0; i < n; i++)
        (5)
    cout << endl;
}
int main()
{
string Str[10] = {"Zhang", "Li", "Wang", "Qian", "Zhao", "Wu", "Xu", "Tang", "Shen", "Liang"};
int Int[8] = {20, 12, 0, -5, 9, -18, 6, 11};
double Dou[7] = {3.07, 8.12, -0.45, 6, 10, -9, 7.29};
Sort(Str, 10);
Sort(Int, 8);
Sort(Dou, 7);
Print(Str, 10);
Print(Int, 8);
Print(Dou, 7);
return 0;
}

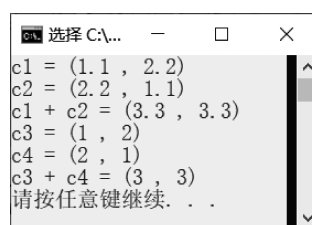
```

2. 以下程序的一次运行结果如图 11.4 所示。请在画线处填入恰当的表达式,使程序完整。

```

#include <iostream>
using namespace std;
    (1) //模板说明
class Complex
{   public:
        Complex(T1 r = 0, T1 i = 0 );
    private:
        T1   Real, Image;
        template< typename T2 >
        friend Complex< T2 > operator + (const Complex< T2 > c1, const Complex< T2 > c2);
        template< typename T3 >
        friend ostream & operator <<(ostream &output, const Complex< T3 > &c );
};
template< typename T >
    (2) //构造函数头
{   Real = r;

```



```

c1 = (1.1 , 2.2)
c2 = (2.2 , 1.1)
c1 + c2 = (3.3 , 3.3)
c3 = (1 , 2)
c4 = (2 , 1)
c3 + c4 = (3 , 3)
请按任意键继续. . .

```

图 11.4 程序填空题 2 运行结果

```

        Image = i;
    }
    template<typename T>
    Complex<T> operator + (const Complex<T> c1,const Complex<T> c2 )
    {
        T r = c1.Real + c2.Real;
        T i = c1.Image + c2.Image;
        return Complex<T>(r,i );
    }
    template<typename T>
    _____(3)_____ //重载<<函数头
    {
        output <<'(' <<c.Real <<" , "<<c.Image <<')';
        return output;
    }
    int main()
    {
        _____(4)_____ c1(1.1,2.2 ),c2(2.2,1.1 ),temp1; //实例化类模板,定义对象
        cout <<"c1 = "<<c1 <<"\nc2 = "<<c2 << endl;
        temp1 = c1 + c2;
        cout <<"c1 + c2 = "<<temp1 << endl;
        _____(5)_____ c3(1,2),c4(2,1),temp2; //实例化类模板,定义对象
        cout <<"c3 = "<<c3 <<"\nc4 = "<<c4 << endl;
        temp2 = c3 + c4;
        cout <<"c3 + c4 = "<<temp2 << endl;
    }

```

3. 使用函数模板实现对不同类型数组求平均值的功能,并在 main()函数中分别输出一个整型数组和一个单精度浮点型数组的平均值。请在画线处填上适当的语句。

```

#include <iostream>
using namespace std;
template <typename T>
double average( _____(1)_____,int size )
{
    T sum = 0;
    for(int i = 0;i < size;i++)
        _____(2)_____
        return sum/size;
}
int main()
{
    int a[ ] = { 1,2,3,4,5,6,7,8,9,10 };
    double b[ ] = { 1.1,2.2,3.3,4.4,5.5,6.6,7.7,8.8,9.9,10 };
    cout <<"Average of array a:"<< _____(3)_____ << endl;
    cout <<"Average of array b:"<< _____(4)_____ << endl;
}

```

四、程序设计题

1. 编写一个模板类,该类包含一个向量成员。提供一个成员函数,用于在已排序的向量中进行二分查找,并返回元素的位置,如果未找到则返回-1。

输入: 包含多组数据,每组数据由三行组成。第一行是一个整数 n,表示向量的大小;第二行包含 n 个整数,表示向量的元素;第三行是一个整数 x,表示要查找的元素。

输出：对于每组输入数据，输出元素的位置。

样例输入：

```
5
1 2 3 4 5
3
```

样例输出：

```
2
```

2. 编写一个函数模板，该函数接收一个字符串参数，并返回该字符串中每种字符出现的次数。

输入：包含多组数据，每组数据是一行字符串。

输出：对于每组输入数据，输出每种字符出现的次数。

样例输入：

```
moonshot
```

样例输出：

```
m:1 o:2 n:1 s:1 h:1 o:1 t:1
```

3. 创建一个模板类，该类包含一个固定大小的数组成员。提供一个成员函数，用于去除数组中的重复元素，并返回去重后的数组。

输入：包含多组数据，每组数据由两行组成。第一行是一个整数 n ，表示数组的大小；第二行包含 n 个整数，表示数组的元素。

输出：对于每组输入数据，输出去重后的数组元素。

样例输入：

```
5
1 2 2 3 4
```

样例输出：

```
1 2 3 4
```

4. 编写一个函数模板，该函数接收一个向量参数，并返回一个新的向量，新向量中的每个元素是原向量对应元素的乘积。

输入：包含多组数据，每组数据由两行组成。第一行是一个整数 n ，表示向量的大小；第二行包含 n 个整数，表示向量的元素。

输出：对于每组输入数据，输出新向量中的元素。

样例输入：

```
3
1 2 3
```

样例输出：

```
6
```

5. 编写一个模板类，该类包含一个字符串成员。提供一个成员函数，用于加密字符串，加密规则是将每个字符的 ASCII 码值加 3。

输入：包含多组数据，每组数据是一行字符串。

输出：对于每组输入数据，输出加密后的字符串。

样例输入：

```
hello
```

样例输出

```
khoor
```

6. 编写一个模板类，该类包含一个向量成员，提供一个成员函数，用于计算向量中所有元素的和。

输入：包含多组数据，每组数据由两行组成。第一行是一个整数 n ，表示向量的大小；第二行包含 n 个整数，表示向量的元素。

输出：对于每组输入数据，输出该向量中所有元素的和。

样例输入

```
3
1 2 3
4
1 2 3 4
```

样例输出：

```
6
10
```

7. 编写一个函数模板，该函数接收一个字符串参数，并返回该字符串的反转版本。

输入：包含多行，每行是一个字符串。

输出：对于每行输入的字符串，输出其反转后的版本。

样例输入：

```
hello
world
```

样例输出：

```
olleh
dlrow
```

8. 创建一个模板类，该类包含一个固定大小的数组成员。提供一个成员函数，用于找出数组中元素的最大值。

输入：包含多组数据，每组数据由两行组成。第一行是一个整数 n ，表示数组的大小；第二行包含 n 个整数，表示数组的元素。

输出：对于每组输入数据，输出该数组中的最大值。

样例输入：

```
5
1 2 3 4 5
3
10 2 8
```

样例输出：

5
10

9. 编写一个函数模板,该函数接收一个向量参数,并返回一个新的向量,新向量中的每个元素是原向量对应元素的平方。

输入: 包含多组数据,每组数据由两行组成。第一行是一个整数 n ,表示向量的大小;第二行包含 n 个整数,表示向量的元素。

输出: 对于每组输入数据,输出新向量中的元素。

样例输入:

4
1 2 3 4

样例输出:

1 4 9 16

10. 编写一个模板类,该类包含一个向量成员。提供一个成员函数,用于过滤向量中的元素,只保留满足特定条件的元素,并返回过滤后的向量。

输入: 包含多组数据,每组数据由三行组成。第一行是一个整数 n ,表示向量的大小;第二行包含 n 个整数,表示向量的元素;第三行是一个整数 k ,表示过滤条件。

输出: 对于每组输入数据,输出过滤后的向量元素。

样例输入:

5
1 2 3 4 5
3

样例输出;

3 4 5